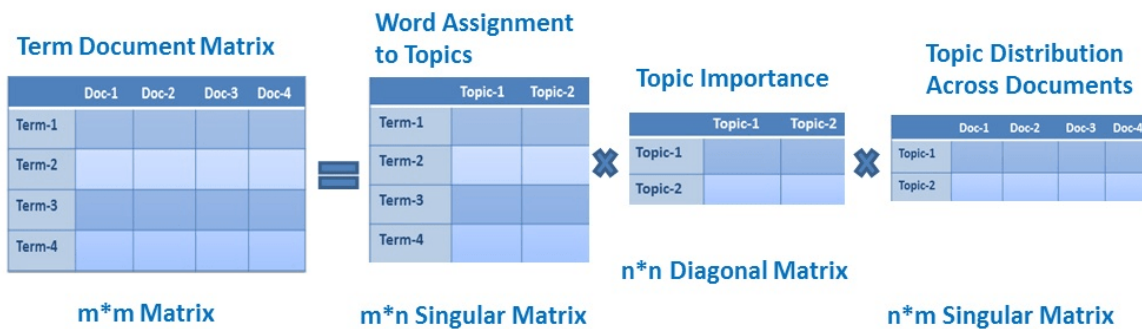

Latent Semantic Indexing (LSI)

Latent Semantic Analysis

LSA (Latent Semantic Analysis) also known as LSI (Latent Semantic Index) LSA uses bag of word(BoW) model, which results in a term-document matrix(occurrence of terms in a document). Rows represent terms and columns represent documents. LSA learns latent topics by performing a matrix decomposition on the document-term matrix using Singular value decomposition. LSA is typically used as a dimension reduction or noise reducing technique.

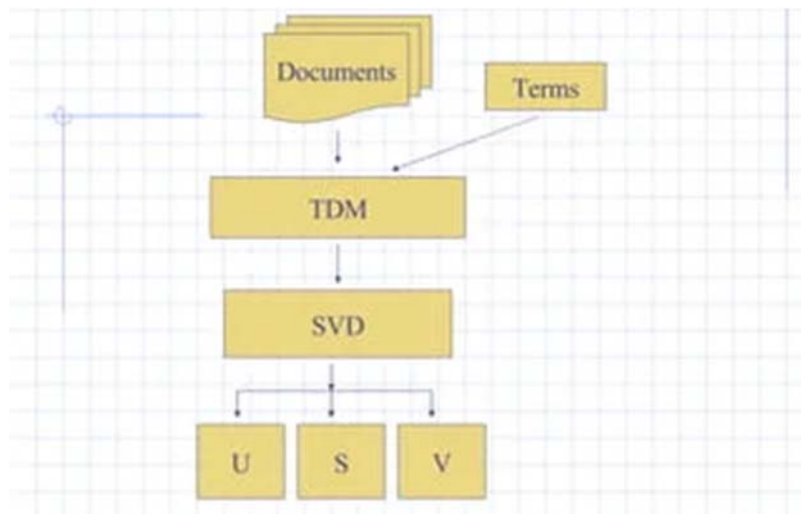


Pros and Cons of LSA

LSA algorithm is the simplest method which is easy to understand and implement. It also offers better results compared to the vector space model. It is faster compared to other available algorithms because it involves document term matrix decomposition only.

Latent topic dimension depends upon the rank of the matrix so we can't extend that limit. LSA decomposed matrix is a highly dense matrix, so it is difficult to index individual dimension. LSA unable to capture the multiple meanings of words. It is not easier to implement compared to LDA (latent Dirichlet allocation). It offers lower accuracy than LDA.

Latent Semantic Indexing (LSI) is a technique used in information retrieval to improve the accuracy and effectiveness of document retrieval by capturing the latent semantic relationships between terms and documents. It addresses the limitations of traditional keyword-based retrieval methods by considering the context and meaning of words in addition to their literal occurrences.



In LSI, documents and terms are represented as vectors in a high-dimensional space, where each dimension corresponds to a unique term in the vocabulary. The key idea behind LSI is to transform this high-dimensional term-document matrix into a lower-dimensional space using Singular Value Decomposition (SVD). This transformation enables LSI to capture the underlying semantic structure of the document collection by identifying latent semantic dimensions that capture the co-occurrence patterns of terms across documents.

The significance of Latent Semantic Indexing in information retrieval systems can be summarized as follows:

1. Semantic Understanding: LSI goes beyond the exact matching of keywords and takes into account the semantic relationships between terms. By analyzing the co-occurrence patterns of terms across documents, LSI can identify synonyms, related terms, and concepts, allowing for a more nuanced understanding of document content.

2. Concept-Based Retrieval: Unlike traditional keyword-based retrieval, which relies solely on exact keyword matches, LSI enables concept-based retrieval. Documents containing semantically related terms or concepts, even if they do not share exact keyword matches, can be retrieved more accurately using LSI.

3. Reduced Sensitivity to Lexical Variations: LSI mitigates the effects of lexical variations, such as synonyms, acronyms, and misspellings, by capturing the underlying semantic similarities between terms. This reduces the risk of retrieval failures due to mismatches between the terms used in queries and those present in documents.

4. Improved Relevance Ranking: By representing documents and queries in a lower-dimensional semantic space, LSI facilitates more effective relevance ranking. Documents that are semantically similar to the query, even if they do not contain exact keyword matches, are ranked higher, leading to improved retrieval precision and recall.

5. Mitigation of Data Sparsity: In large document collections, many terms may have sparse occurrences, making it challenging to accurately estimate term-document associations. LSI reduces data sparsity by capturing the shared semantic information among terms and documents, thereby improving retrieval performance, particularly for rare or long-tail terms.

6. Robustness to Noise: LSI is inherently robust to noise and irrelevant terms in the document collection. By focusing on the most significant semantic dimensions captured by SVD, LSI

effectively filters out noise and irrelevant variations in the data, leading to more reliable retrieval results.

Explain the architecture and functioning of an LSI-based search engine. Discuss how LSI is used to rank documents based on their semantic relevance to a query.

Latent Semantic Indexing (LSI) is a technique used in information retrieval to overcome the limitations of traditional keyword-based search engines by capturing the underlying semantic relationships between terms and documents. The architecture and functioning of an LSI-based search engine involve several key components:

1. Document Collection: The search engine starts with a collection of documents, each represented as a vector in a high-dimensional space, where each dimension corresponds to a unique term in the vocabulary.

2. Term-Document Matrix: A term-document matrix is constructed to represent the frequency of each term in each document. However, instead of using raw term frequencies, LSI typically applies some form of term weighting (e.g., TF-IDF) to account for the importance of terms in distinguishing between relevant and irrelevant documents.

3. Singular Value Decomposition (SVD): The term-document matrix is then decomposed using Singular Value Decomposition (SVD) into three matrices: U , Σ , and V . The matrix Σ contains the singular values, which represent the importance of each dimension in capturing the underlying semantic structure of the document collection.

4. Dimensionality Reduction: In LSI, only the top k singular values and their corresponding columns in U and V are retained, where k is a user-defined parameter typically chosen based on cross-validation or other optimization techniques. This process effectively reduces the dimensionality of the term-document space while preserving the most significant semantic information.

5. Semantic Space Representation: The reduced-dimensional space, often referred to as the latent semantic space, captures the latent semantic relationships between terms and documents. Each document and query are now represented as vectors in this semantic space.

6. Ranking of Documents: When a user submits a query, the search engine first represents the query in the latent semantic space. Then, it calculates the cosine similarity between the query vector and each document vector to measure their semantic relevance. Documents with higher cosine similarity scores are considered more semantically relevant to the query and are ranked higher in the search results.

Comparison with Traditional Keyword-based Search Engines:

1. Semantic Relevance vs. Exact Match: Traditional keyword-based search engines rely on exact keyword matching to retrieve documents, which may lead to issues such as synonymy, polysemy, and poor retrieval of semantically related documents. In contrast, LSI search engines consider the semantic similarity between terms and documents, allowing them to retrieve relevant documents even if they do not contain exact keyword matches.

2. Robustness to Vocabulary Variations: LSI search engines are more robust to variations in vocabulary, spelling errors, and linguistic nuances compared to traditional keyword-based search engines. They can capture the underlying semantic meaning of terms and recognize documents that use different terms to express similar concepts.

3. Scalability and Efficiency: While LSI-based search engines require additional computational overhead for performing SVD and representing documents in the latent semantic space, they can scale efficiently to large document collections. Traditional keyword-based search engines may struggle with scalability due to the need to match each query term against the entire document collection.

In summary, LSI-based search engines offer improved retrieval accuracy by considering the semantic relationships between terms and documents, making them a valuable alternative to traditional keyword-based search engines, especially in scenarios where precise keyword matches are not sufficient for capturing document relevance.

Probabilistic analysis of Latent Semantic Indexing (LSI)

The probabilistic analysis of Latent Semantic Indexing (LSI) provides a theoretical foundation for understanding the effectiveness of LSI in capturing the latent semantic structure of a document collection. This analysis builds upon probabilistic models to describe the relationships between terms and documents in the context of information retrieval. Here's an overview of the probabilistic analysis of LSI:

1. Probabilistic Information Retrieval Models:

Probabilistic Information Retrieval Models, such as the Vector Space Model (VSM) and the Probabilistic Latent Semantic Analysis (PLSA), form the basis for understanding the probabilistic analysis of LSI. These models represent documents and queries as vectors in a high-dimensional space and use probabilistic measures, such as term probabilities and document likelihoods, to rank documents based on their relevance to a given query.

2. Document Generation Process:

In the probabilistic analysis of LSI, documents are assumed to be generated from a probabilistic process. This process involves selecting words from a distribution over terms conditioned on the underlying semantic topics or dimensions. Each document is generated by sampling words from these topic distributions, resulting in a probabilistic representation of the document's content.

3. Latent Semantic Dimensions:

LSI assumes that documents and terms are related through latent semantic dimensions or topics. These latent dimensions capture the underlying themes or concepts present in the document collection. In the probabilistic analysis, these dimensions are represented as probability distributions over terms, where each dimension corresponds to a semantic topic.

4. Term and Document Probability Distributions:

In the probabilistic analysis of LSI, the goal is to model the joint probability distribution of terms and documents. This involves estimating the probability of observing a particular term given a document ($P(\text{term}|\text{document})$) and the probability of observing a document given a term ($P(\text{document}|\text{term})$). These probability distributions reflect the likelihood of terms and documents co-occurring in the context of the underlying semantic dimensions.

5. Singular Value Decomposition (SVD) Interpretation:

The probabilistic analysis of LSI provides insights into the interpretation of Singular Value Decomposition (SVD) in terms of probabilistic concepts. SVD decomposes the observed term-document matrix into a set of orthogonal basis vectors (singular vectors) and corresponding singular values. In the probabilistic framework, these singular vectors can be interpreted as representing the latent semantic dimensions, while the singular values indicate the importance or relevance of each dimension in capturing the variability of the document collection.

6. Relevance Ranking and Query Likelihood:

Based on the probabilistic analysis, LSI calculates the relevance of documents to a given query by measuring the likelihood of observing the query terms given the documents' probabilistic representations. This approach, known as query likelihood estimation, combines term probabilities and document likelihoods to rank documents according to their semantic relevance to the query.

In summary, the probabilistic analysis of LSI offers a principled framework for understanding how LSI captures the latent semantic structure of a document collection. By modeling the probabilistic relationships between terms and documents, this analysis provides insights into the effectiveness of LSI in information retrieval and its interpretation in terms of probabilistic concepts.

Discuss theoretical frameworks used to analyze the effectiveness of LSI in capturing semantic relationships between terms and documents.

The probabilistic analysis of Latent Semantic Indexing (LSI) involves theoretical frameworks aimed at understanding and quantifying the effectiveness of LSI in capturing semantic relationships between terms and documents. Several approaches have been proposed to analyze LSI from a probabilistic perspective, each offering insights into the underlying mechanisms of LSI and its impact on information retrieval. Here, we'll discuss two main theoretical frameworks used in the probabilistic analysis of LSI:

1. Probabilistic Latent Semantic Analysis (pLSA):

- Probabilistic Latent Semantic Analysis is a probabilistic model that extends LSI by introducing a generative probabilistic framework. It assumes that documents are generated from a mixture of latent topics, and each topic is characterized by a distribution over terms.

- The key idea behind pLSA is to model the joint probability distribution of terms and documents, with latent topics serving as hidden variables. It uses the Expectation-Maximization (EM) algorithm to estimate the parameters of the model, including the topic distributions and the topic proportions for each document.

- pLSA provides a probabilistic interpretation of LSI, allowing researchers to derive statistical measures of relevance and assess the uncertainty associated with document rankings. It has been used to analyze the effectiveness of LSI in capturing semantic relationships and to explore extensions of LSI, such as incorporating document priors and handling out-of-vocabulary terms.

2. Probabilistic Latent Semantic Indexing (pLSI):

- Probabilistic Latent Semantic Indexing is a variant of LSI that incorporates a probabilistic generative model similar to pLSA. It assumes a probabilistic generative process for generating the observed term-document matrix, where each document is represented as a mixture of latent topics, and each term is associated with a specific topic.

- In pLSI, the parameters of the generative model are estimated using the Expectation-Maximization algorithm, similar to pLSA. However, unlike pLSA, pLSI directly models the observed term-document matrix without introducing additional latent variables.

- pLSI provides a probabilistic interpretation of LSI that enables the estimation of uncertainty in document rankings and the computation of probabilistic relevance scores. It has been used to analyze the properties of LSI and to investigate the impact of different model assumptions on retrieval performance.

Both pLSA and pLSI offer valuable insights into the probabilistic properties of LSI and provide theoretical frameworks for analyzing its effectiveness in capturing semantic relationships between terms and documents. By modeling the uncertainty inherent in the document-ranking process, these probabilistic approaches enhance our understanding of LSI and its applications in information retrieval.

Identify potential challenges and limitations of Latent Semantic Indexing, such as sensitivity to noise and computational overhead.

Latent Semantic Indexing (LSI) offers significant advantages in capturing semantic relationships between terms and documents, but it also faces several challenges and limitations, including:

1. Sensitivity to Noise:

- LSI relies on the assumption that the underlying semantic structure of the document collection can be effectively captured by reducing the dimensionality of the term-document matrix. However, noisy or irrelevant terms in the document collection can distort the latent semantic space, leading to degraded retrieval performance.

- Noise in the form of misspelled words, grammatical errors, or irrelevant terms can affect the accuracy of term-document associations and diminish the effectiveness of LSI in identifying relevant documents.

2. Computational Overhead:

- The process of performing Singular Value Decomposition (SVD) to reduce the dimensionality of the term-document matrix can be computationally intensive, especially for large document collections. As the size of the document collection increases, the computational overhead associated with SVD becomes a significant bottleneck.

- Updating LSI models to incorporate new documents or queries also incurs computational costs, as it requires re-computation of the SVD or incremental updating techniques, which can be complex and time-consuming.

3. Interpretability and Transparency:

- LSI transforms the original term-document space into a lower-dimensional semantic space, which can make the interpretation of the resulting document representations challenging. While LSI improves retrieval effectiveness by capturing semantic relationships, it may not provide transparent explanations for why certain documents are ranked higher than others.

- Users may find it difficult to interpret the semantic dimensions extracted by LSI or understand the reasons behind document rankings, leading to reduced trust and usability of the retrieval system.

4. Limited Handling of Polysemy and Synonymy:

- LSI may struggle to effectively disambiguate terms with multiple meanings (polysemy) or to recognize terms that have similar meanings (synonymy). Since LSI relies on co-occurrence statistics to capture term relationships, it may not fully address these linguistic challenges.

- While LSI can partially mitigate polysemy and synonymy by clustering related terms in the latent semantic space, it may not always provide precise distinctions between different senses of a term or accurately capture fine-grained semantic nuances.

5. Scalability Issues:

- As the size of the document collection grows, the computational and memory requirements of LSI-based retrieval systems may become prohibitive. Storing and processing large term-document matrices and performing SVD on massive datasets can strain computational resources and limit the scalability of LSI to very large document collections.

- Scalability concerns may arise not only during the initial indexing phase but also during query processing, especially in real-time or interactive retrieval scenarios where low-latency responses are required.

Addressing these challenges requires ongoing research and development efforts to enhance the robustness, efficiency, and interpretability of LSI-based retrieval systems. Techniques such as noise reduction, incremental updating, and incorporating domain-specific knowledge can help mitigate some of the limitations of LSI and improve its effectiveness in practical information retrieval applications.

Discuss emerging research directions aimed at addressing these challenges and advancing the effectiveness of LSI-based retrieval methods.

Emerging research directions aim to address the challenges and limitations of Latent Semantic Indexing (LSI) while advancing its effectiveness in information retrieval. Several promising approaches are being explored to enhance the robustness, efficiency, and interpretability of LSI-based retrieval methods:

1. Deep Learning-Based Models:

- Deep learning techniques, such as neural networks, offer powerful tools for learning complex representations of text data. Researchers are exploring the use of deep learning architectures, such as convolutional neural networks (CNNs) and recurrent neural networks (RNNs), to automatically learn latent semantic representations from raw text data.

- By leveraging deep learning models, researchers aim to capture more nuanced semantic relationships between terms and documents, addressing limitations associated with the handcrafted feature engineering in traditional LSI approaches. Deep learning-based models have shown promising results in tasks such as semantic similarity estimation and document ranking.

2. Word Embeddings and Semantic Embeddings:

- Word embeddings, such as Word2Vec and GloVe, represent words as dense, low-dimensional vectors in a continuous semantic space. These embeddings capture rich semantic information based on the distributional properties of words in large text corpora.

- Researchers are investigating methods to incorporate word embeddings into LSI-based retrieval systems to improve the representation of terms and documents in the latent semantic space. By leveraging pre-trained word embeddings or jointly learning embeddings during the LSI process, these approaches aim to enhance the semantic coherence and discriminative power of LSI representations.

3. Hybrid Models and Ensemble Methods:

- Hybrid models combine LSI with other information retrieval techniques, such as probabilistic models or graph-based algorithms, to leverage complementary strengths and mitigate individual weaknesses. For example, combining LSI with probabilistic models like Latent Dirichlet Allocation (LDA) can capture both global and local semantic structures in the document collection.

- Ensemble methods aggregate predictions from multiple retrieval models, including LSI, to improve robustness and performance. By combining diverse retrieval strategies, such as LSI, term-

based models, and neural network-based models, ensemble methods aim to achieve superior retrieval effectiveness across diverse datasets and retrieval tasks.

4. Interpretable Latent Spaces:

- Researchers are exploring techniques to enhance the interpretability of latent semantic spaces learned by LSI. By incorporating domain-specific constraints, such as hierarchical topic structures or semantic relationships, into the LSI model, researchers aim to create more interpretable representations of terms and documents.

- Methods for visualizing and exploring latent semantic spaces enable users to gain insights into the underlying semantic structures captured by LSI. Interactive visualization tools, dimensionality reduction techniques, and clustering algorithms help users interpret and navigate complex semantic spaces, enhancing the usability and transparency of LSI-based retrieval systems.

5. Efficient Algorithms and Scalable Implementations:

- To address scalability concerns associated with LSI-based retrieval systems, researchers are developing efficient algorithms and scalable implementations optimized for large-scale document collections. Distributed computing frameworks, parallel processing techniques, and approximate SVD algorithms enable LSI to handle massive datasets efficiently.

- Streamlined indexing procedures, incremental updating algorithms, and distributed storage solutions facilitate real-time indexing and querying in dynamic environments. These advancements ensure that LSI-based retrieval systems remain scalable and responsive to the needs of modern information retrieval applications.

By pursuing these emerging research directions, researchers aim to overcome the challenges of Latent Semantic Indexing and unlock its full potential for semantic search, document clustering, recommendation systems, and other information retrieval tasks. These efforts contribute to advancing the state-of-the-art in information retrieval and facilitating the development of more effective and user-friendly retrieval systems.

Singular Value Decomposition(SVD)

SVD is a matrix factorization method that represents a matrix in the product of two matrices. It offers various useful applications in signal processing, psychology, sociology, climate, and atmospheric science, statistics and astronomy.

$$M=U\Sigma V^*$$

- M is an $m \times m$ matrix
- U is a $m \times n$ left singular matrix / Orthogonal matrix
- Σ is a $n \times n$ diagonal matrix with non-negative real numbers.
- V is a $m \times n$ right singular matrix / Orthogonal matrix
- V^* is $n \times m$ matrix, which is the transpose of the V .

Identity matrix: It is a square matrix in which all the elements of the principal diagonal are ones, and all other elements are zeros.

Diagonal Matrix: It is a matrix in which the entries other than the main diagonal are all zero.

Singular Matrix: A matrix is singular if its determinant is 0 or a square matrix that does not have a matrix inverse.

- It best for google search engine to identify the search results by using singular value decomposition technique, It also used in Face recognition software's as well for better result.
- Let us assume we have $A_{m \times n}$ matrix, let us try to decompose the A matrix using SVD
- We can decompose it in three different simple matrix
- So let us write :

$$A = U \Sigma V^T$$

- Where Matrix U and V both are orthogonal Matrix
- Where Σ Matrix is diagonal matrix. It carries the singular values and it is arranged in descending order like $\sigma_1 \geq \sigma_2 \geq \sigma_3 \geq \dots \geq \sigma_r$
- Where "r" is the rank of matrix A

Let us apply SVD on A Matrix

$$\begin{aligned} A^T A &= (U \Sigma V^T)^T (U \Sigma V^T) \\ &= V \Sigma^T U^T U \Sigma V^T \\ &= \text{The U matrix is an orthogonal matrix so it is identical. IT is a property of orthogonaol matrix that if we multiply with its transpose the result would be identical.} \\ &= V \Sigma^T \Sigma V^T \\ &= \text{Sigma is a diagonal matrix so the transpose of the diagonal matrix is same like sigma.} \end{aligned}$$

$$A^T A = V \Sigma^2 V^T \quad \text{It is the form of diagonalization}$$

1. So the important part is matrix V carries the eigen vectors of $A^T A$

Now if we squareroot of $A^T A$ then

$$\sqrt{A^T A} = V \Sigma V^T$$

2. So it stat that now Σ carries the eigen values of $\sqrt{A^T A}$

So by using the 1. And 2. We can find the Σ value

Next Step

- Let us find the U matrix now

$$\begin{aligned} A A^T &= (U \Sigma V^T) (U \Sigma V^T)^T \\ &= U \Sigma^T \Sigma V^T V U \end{aligned}$$

= The V matrix is an orthogonal matrix so it is identical. IT is a property of orthogonaol matrix that if we multiply with its transpose the result would be identical.

$= U \Sigma^T \Sigma U^T$
 $= \text{Sigma is a diagonal matrix so the transpose of the diagonal matrix is same like sigma.}$
 $AA^T = U \Sigma^2 U^T$ It is the form of diagonalization
 3. So the important part is matrix U carries the eigen vectors of AA^T

Conclusion:

Objective is to find SVD of $A_{m \times n}$ so $A = U \Sigma V^T$

$V_{n \times n}$	V contains the eigen vectors of $A^T A$	[v is orthogonal matrix]
$\Sigma_{m \times n}$	Σ Contain the eigen values of $\sqrt{A^T A}$	[Σ is diagonal matrix]
$U_{m \times m}$	U contains the eigen vectors of AA^T	[U is orthogonal matrix]

Example:

Find SVD of $\begin{pmatrix} 1 & 0 & 1 \\ 0 & -1 & 0 \end{pmatrix}_{2 \times 3}$

Solution:

Initially let us find the value of $A^T A$

So: $A = U \Sigma V^T$

$$A^T A = \begin{pmatrix} 1 & 0 \\ 0 & -1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 & 1 \\ 0 & -1 & 0 \end{pmatrix}$$

Let us multiply A^T rows with A columns

$$A^T A = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}$$

Now let us find AA^T

$$AA^T = S \lambda S^{-1}$$

We have to find eigen vectors and eigen values so

$$\lambda_1 = 1, \quad \vec{x}_1 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \quad \text{For more details check the properties of } \lambda \text{ eigen values and eigen vector } x$$

$$\lambda_2 = 0, \quad \vec{x}_2 = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}; -c_1 + 0 \times c_2 + c_3$$

$$\lambda_3 = \text{trace} - (\lambda_1 + \lambda_2); \text{ where trace is the sum of diagonal}$$

$$= 3 - (1 + 0) = 2$$

Now to find last x eigen vector use the formula:

$$(A - \lambda_I) \vec{x} = 0$$

$$\vec{x}_3 = \begin{bmatrix} 1-2 & 0 & 1 \\ 0 & 1-2 & 0 \\ 1 & 0 & 1-2 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \vec{0}$$

$$= \begin{bmatrix} -1 & 0 & 1 \\ 0 & -1 & 0 \\ 1 & 0 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$R_3 = R_3 + R_1$$

$$= \begin{bmatrix} -1 & 0 & 1 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Let

$$c = 1$$

$$b = 0$$

$$-a + c = 0$$

$$a = c$$

$$a = 1$$

so

$$\vec{x_3} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$

Finally we got all the eigen vectors and eigen values of $A^T A$ matrix, so we can calculate the value of V and Σ

Now to calculate V we need all the eigen vectors, However, they must be normalized.

But as per result all the eigen vectors are not normalized.

So let us normalize

Step 1.

$$\vec{x_1} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} ; \text{ it remain the same as it is normalized}$$

Step 2.

$$\vec{x_2} = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} ; \text{ this matrix is not normalized so let us divide by } 2$$

$$\vec{x_2} = \frac{1}{\sqrt{2}} \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix}$$

Step 3.

Same with $\vec{x_3}$

$$\vec{x_3} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$

So the V eigen vectors are as follows:

- Where the Σ values must be sorted and in decending order, so the eigen vectors also in sorted order and normalized
- As λ_3 value is the bigger one i.e 2; so first we have to place the value of $\vec{x_3}$

So we have to place it like ($\vec{x_3}$ $\vec{x_1}$ $\vec{x_2}$)

$$V = \begin{pmatrix} 1/\sqrt{2} & 0 & -1/\sqrt{2} \\ 0 & 1 & 0 \\ 1/\sqrt{2} & 0 & 1/\sqrt{2} \end{pmatrix}$$

Next lets calculate the value of Σ which contain the eigen values of $\sqrt{A^T A}$

As Σ is a diagonal matrix so to find the squareroot is easy i.e the value of the diagonal value's squareroot is the actual value.

$$\Sigma = \begin{pmatrix} \sqrt{2} & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

The last part we have to find U matrix

So

U = eigen vectors of AA^T

$$U = \begin{pmatrix} 1 & 0 & 1 \\ 0 & -1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & -1 \\ 1 & 0 \end{pmatrix}$$

$$= \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$$

Now to find the eigen values of U

So

$$\lambda_1 = 2$$

$$\lambda_2 = 1$$

So finally the eigen values of $A^T A$ is equal to the eigen values of AA^T

So the eigen vector are:

$$\lambda_1 = 2 ; \rightarrow x_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$\lambda_2 = 1 ; \rightarrow x_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

So; as the value of λ_1 is greater so we have to place its vector value on the first position

$$U = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

We can verify the results by placing the values in A matrix and check LHS = RHS hence proved.

References:

1. https://en.wikipedia.org/wiki/Latent_semantic_analysis#:~:text=LSI%20uses%20common%20linear%20algebra,concepts%20contained%20in%20the%20text.
2. https://www.geeksforgeeks.org/latent-semantic-analysis/?ref=ml_lbp
3. https://en.wikipedia.org/wiki/Singular_value_decomposition
4. [https://www.geeksforgeeks.org/singular-value-decomposition-svd/#:~:text=The%20Singular%20Value%20Decomposition%20\(SVD,important%20applications%20in%20data%20science.](https://www.geeksforgeeks.org/singular-value-decomposition-svd/#:~:text=The%20Singular%20Value%20Decomposition%20(SVD,important%20applications%20in%20data%20science.)